SHERLOCK

Security Review For Maple



Collaborative Audit Prepared For: **Maple**
Lead Security Expert(s): **0x73696d616f**
0xeix
vinica_boy
Date Audited: **September 7 - September 18, 2026**

Introduction

The release ships the Strategy Manager (along with the Strategy Deployer and a permissioned Liquidity Router), the modular Aave V3 and Sky strategies, the new Allocation Strategy, plus upgrades to the Pool Manager and Maple Globals contracts to support the updated system.

A new Strategy Manager contract sits between pools and a protocol-specific underlying strategy, letting multiple Pool Managers co-fund one shared strategy and share profit and loss proportionally via internal (non-tokenised) share accounting, with per-pool and per-strategy exposure caps, and an asynchronous withdrawal workflow.

Share accounting, AUM reporting and caps are identical for every strategy and live in the Strategy Manager; protocol-specific logic stays in the underlying strategy, so new venues are added by writing a new underlying strategy. Strategy Manager depositors are always Pool Managers, never LPs, so KYC, permissioning and redemption mechanics remain in the existing Pool contracts.

Scope

Repository: maple-labs/allocation-strategy

Audited Commit: 244846696b95e2cb6ffe0737eb61afe50193dfe2

Final Commit: d509bd5d7036c5af9a5831487e70f8608e7cc2d1

Files:

- contracts/interfaces/IMapleAllocationStrategyInitializer.sol
- contracts/interfaces/IMapleAllocationStrategy.sol
- contracts/interfaces/IMapleAllocationStrategyStorage.sol
- contracts/interfaces/Interfaces.sol
- contracts/MapleAllocationStrategyFactory.sol
- contracts/MapleAllocationStrategyInitializer.sol
- contracts/MapleAllocationStrategy.sol
- contracts/MapleAllocationStrategyStorage.sol

Repository: maple-labs/defi-strategies

Audited Commit: 263528c52d1b9c3347eb4b41707d57d08e9c4ff3

Final Commit: e889e413d1c96146b371e1474ce18fea8c37e404

Files:

- contracts/aaveV3/MapleAaveV3StrategyFactory.sol

- contracts/aaveV3/MapleAaveV3StrategyInitializer.sol
- contracts/aaveV3/MapleAaveV3Strategy.sol
- contracts/aaveV3/MapleAaveV3StrategyStorage.sol
- contracts/interfaces/IMapleAaveV3StrategyInitializer.sol
- contracts/interfaces/IMapleAaveV3Strategy.sol
- contracts/interfaces/IMapleAaveV3StrategyStorage.sol
- contracts/interfaces/IMapleSkyStrategyInitializer.sol
- contracts/interfaces/IMapleSkyStrategy.sol
- contracts/interfaces/IMapleSkyStrategyStorage.sol
- contracts/interfaces/Interfaces.sol
- contracts/sky/MapleSkyStrategyFactory.sol
- contracts/sky/MapleSkyStrategyInitializer.sol
- contracts/sky/MapleSkyStrategy.sol
- contracts/sky/MapleSkyStrategyStorage.sol

Repository: maple-labs/globals-v2

Audited Commit: a56ea201f3ef0b89c6bb33ac096ad9f98819a0f6

Final Commit: a56ea201f3ef0b89c6bb33ac096ad9f98819a0f6

Files:

- contracts/interfaces/IMapleGlobals.sol
- contracts/MapleGlobals.sol

Repository: maple-labs/maple-core-v2

Audited Commit: b55685a7d0f07b640fbc2e53d5db9c5ae715a3b1

Final Commit: b55685a7d0f07b640fbc2e53d5db9c5ae715a3b1

Files:

- scripts/ModularReleaseDeployment.s.sol

Repository: maple-labs/pool-v2

Audited Commit: 7bc288b1906fa28c26cef2b0c99d99f6b5f186f2

Final Commit: a3f77cc4be6d100d02987b500059432cd6c36e3e

Files:

- contracts/interfaces/IMaplePoolDeployer.sol
- contracts/interfaces/IMaplePoolManagerInitializer.sol
- contracts/interfaces/IMaplePoolManager.sol
- contracts/interfaces/IMaplePoolManagerStorage.sol
- contracts/interfaces/IMaplePool.sol
- contracts/MaplePoolDeployer.sol
- contracts/MaplePoolManager.sol
- contracts/MaplePool.sol
- contracts/proxy/MaplePoolManagerInitializer.sol
- contracts/proxy/MaplePoolManagerMigratorV500.sol
- contracts/proxy/MaplePoolManagerStorage.sol

Repository: maple-labs/strategy-manager

Audited Commit: f278958671c319dc35d0e9499810c13cc18c60c2

Final Commit: dc8aac619aac99d5d52f6fc42421140851bef089

Files:

- contracts/interfaces/IMapleLiquidityRouter.sol
- contracts/interfaces/IMapleStrategyDeployer.sol
- contracts/interfaces/IMapleStrategyManagerInitializer.sol
- contracts/interfaces/IMapleStrategyManager.sol
- contracts/interfaces/IMapleStrategyManagerStorage.sol
- contracts/interfaces/Interfaces.sol
- contracts/MapleLiquidityRouter.sol
- contracts/MapleStrategyDeployer.sol
- contracts/MapleStrategyManagerFactory.sol
- contracts/MapleStrategyManagerInitializer.sol
- contracts/MapleStrategyManager.sol
- contracts/MapleStrategyManagerStorage.sol

Repository: maple-labs/withdrawal-manager-queue

Audited Commit: 76a8ba7e0a7c51a312df395593c71ee01881b16e

Final Commit: b3480a65c9bffc1be2902b1e98e96dd719125256

Files:

- contracts/MapleWithdrawalManager.sol
- contracts/interfaces/IMapleWithdrawalManager.sol
- contracts/interfaces/Interfaces.sol

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	4

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: `MapleStrategyManager::convertToExitShares()` rounds down, so it returns fewer shares than expected [RESOLVED]

Source: https://github.com/sherlock-audit/2026-09-maple-sept-7th-2026/issues/50

Summary

`MapleStrategyManager::convertToExitShares()` returns how many shares are required to exit a given number of assets, rounding down. `processRedemption()` converts the sent shares by calling `_convertToExitAssets()`, which rounds down again, so the redemption pays out less than the input assets initially sent to `MapleStrategyManager::convertToExitShares()`.

Vulnerability Detail

As an example, with `totalShares` equal to `1_000_000e6` and exit assets of `1_500_000e6`, converting 100 USDC gives:

`100e6 * 1_000_000e6 / 1_500_000e6 = 66_666_666`

and redeeming `66_666_666` shares pays 1 less than the initial 100e6 USDC:

`66_666_666 * 1_500_000e6 / 1_000_000e6 = 99_999_999`

Impact

Bots calling `MapleStrategyManager::convertToExitShares()` expecting to get the shares required to redeem the input assets will get less assets when the redemption goes through.

Code Snippet

```
function _convertToExitShares(uint256 assets_, uint256 totalAssets_) internal view
↪  returns (uint256 shares_) {
    shares_ = _convertToShares(assets_, totalAssets_ -
    ↪  _unrealizedLosses(totalAssets_));
}

function _convertToShares(uint256 assets_, uint256 totalAssets_) internal view
↪  returns (uint256 shares_) {
    uint256 totalShares_ = totalShares;

    if (totalShares_ == 0) return assets_;
```

```
    shares_ = assets_ * totalShares_ / totalAssets_;
}
```

https://github.com/sherlock-audit/2026-09-maple-sept-7th-2026/blob/main/maple-labs__strategy-manager-private/contracts/MapleStrategyManager.sol#L398-L404

Tool Used

Manual Review

Recommendation

Round up in `_convertToExitShares()` using `_divRoundUp()`:

```
function _convertToExitShares(uint256 assets_, uint256 totalAssets_) internal view
↪   returns (uint256 shares_) {
    uint256 exitAssets_  = totalAssets_ - _unrealizedLosses(totalAssets_);
    uint256 totalShares_ = totalShares;

    if (totalShares_ == 0) return assets_;

    shares_ = _divRoundUp(assets_ * totalShares_, exitAssets_);
}
```

Discussion

0xfarhaan

fixed in https://github.com/maple-labs/strategy-manager/commit/b0e729438be5dd9d960bb060476b2919d26e1459

0xsimao

Fix in #b0e7294 looks good, now whoever calls `MapleStrategyManager::convertToExitShares()` gets the right amount of shares needed to redeem the sent assets.

Issue L-2: Cancelling a queued redemption after cash was sourced from an impaired strategy moves the exit loss onto the other LPs [RESOLVED]

Source: https://github.com/sherlock-audit/2026-09-maple-sept-7th-2026/issues/51

Summary

`MapleStrategyManager::processRedemption()` burns shares at the exit rate without reducing `impairedAssets`, and `MapleWithdrawalManager::removeSharesById()` lets the request owner cancel at any point before `processRedemptions()`, so the value forfeited to fund one LP's exit is carried by every LP in the pool instead of the LP who requested it.

Vulnerability Detail

Pricing the strategy manager exit net of the impairment is intended: the pool giving up shares carries its part of the loss, and the queue then pays the exiting LP at the pool's own impaired exit price, so the forfeit lands on the LP whose request caused it. What is missing is any coupling between the two steps. `processRedemption()` is the Strategy Delegate's transaction on the strategy manager, `processRedemptions()` is the redeemer's transaction on the withdrawal manager, and neither checks that the request the cash was raised for still exists.

```
uint256 assets_ = _convertToExitAssets(shares_, assetsUnderManagement());

require(assets_ != 0,                      "MSM:PR:ZERO_ASSETS");
require(assets_ <= availableToWithdraw(), "MSM:PR:NOT_ENOUGH_LIQUIDITY");

pendingSharesOf[poolManager_] -= shares_;
totalPendingShares            -= shares_;

sharesOf[poolManager_] -= shares_;
totalShares            -= shares_;
```

The shares are burned at the exit rate but the strategy's `impairedAssets` is unchanged, so the difference stays in the strategy and is spread over every remaining share, most of them the other pool's. That cross-pool transfer happens here, whatever the queue does afterwards.

Between the two transactions the request owner can call `removeSharesById()`, which requires only that the caller owns the request. The pool then holds cash it sold strategy manager shares for at a discount, and the LP is still a depositor.

Consider the following example. Strategy AUM 1000 with 200 impaired, Pool A holds 600 shares and Pool B 400. Pool B redeems 100 shares to serve a request from an LP holding a

quarter of the pool:

```
assets = 100 * (1000 - 200) / 1000 = 80
```

20 stays in the strategy over the 900 remaining shares. At gross valuation Pool A is now `600 * 920 / 900 = 613.33` and Pool B is `300 * 920 / 900 + 80 = 386.67`. Exit valuations are unchanged, so nothing is visible while the impairment stands. The LP cancels and the impairment is later removed:

- Pool A: 600, now 613.33.
- Pool B: 400, now 386.67, with nobody having exited. The LP who requested pays 3.33 of it, the other LPs pay 10.

Without the cancel the exiting LP is paid 80 for shares worth 100 at gross and carries the whole 20. Only if the loss is realised at exactly the impaired amount is nobody worse off.

Impact

LPs who did not request a withdrawal pay, pro rata, for another LP's exit from an impaired strategy once the impairment is removed. The other pool's gain is the same with or without the cancel. Note: this requires the Strategy Delegate to process a redemption while the strategy is impaired, since bots and the router are already locked out while `impairedAssets != 0`.

Code Snippet

https://github.com/maple-labs/strategy-manager/blob/f278958671c319dc35d0e9499810c13cc18c60c2/contracts/MapleStrategyManager.sol#L250-L279

https://github.com/maple-labs/withdrawal-manager-queue/blob/76a8ba7e0a7c51a312df395593c71ee01881b16e/contracts/MapleWithdrawalManager.sol#L279-L295

https://github.com/maple-labs/withdrawal-manager-queue/blob/76a8ba7e0a7c51a312df395593c71ee01881b16e/contracts/MapleWithdrawalManager.sol#L205-L232

Tool Used

Manual Review

Recommendation

Consider processing the strategy manager redemption and the withdrawal queue in one transaction whenever liquidity is sourced from an impaired strategy.

As discussed with the team, the operational admin will be able to process the redemption from an impaired strategy. It already passes the withdrawal manager's `onlyR`

`edeemer`, so it can batch `processRedemption()` and `processRedemptions()` in one transaction. The LP is then paid at the impaired price in the same transaction that burns the strategy manager shares, and there is no point at which the cash is sourced and the request can still be cancelled.

`processRedemptions()` skips a cancelled request without reverting, so a cancel that lands just before the batch still leaves the strategy manager redemption executed. To rule that out, the operational admin can clear the requester's permission in `MaplePoolPermissionManager` before sending the batch and restore it after. `removeSharesById()` returns shares through `MaplePool::transfer()`, which then reverts.

Discussion

0xfarhaan

We have added the operational admin to be able to process redemptions on the strategy manager as fixed in this PR https://github.com/maple-labs/strategy-manager/commit/dc8aac619aac99d5d52f6fc42421140851bef089

As per the recommendation this will be dealt with operationally. Initially non impaired strategies will be used for liquidity to facilitate LP withdrawals. In the case the impaired strategy is the last liquidity source with LPs willing to withdraw an atomic batch will be created to set permissions and process withdrawals across the strategy and pool before restoring permissions.

Additionally we have updated the WM removeSharesById() function to include the standard pool permission manager check we have across LP facing functions which will give Maple the ability to pause redemption cancellations to avoid front running. https://github.com/maple-labs/withdrawal-manager-queue/commit/b3480a65c9bffc1be2902b1e98e96dd719125256

Issue L-3: `setMetadata` NatSpec deviates from the code that allows the strategy delegate [RESOLVED]

Source: https://github.com/sherlock-audit/2026-09-maple-sept-7th-2026/issues/52

Summary

Aave and Sky `setMetadata` NatSpec states the function is callable only by protocol admins. The implementation allows the strategy delegate or the operational admin.

Vulnerability Detail

IMapleAaveV3Strategy.setMetadata documents:

```
/**
 *  @dev   Updates the metadata of the Aave V3 strategy.
 *         Callable only by protocol admins.
 *  @param metadata Optional metadata associated with the strategy.
 */
function setMetadata(string calldata metadata) external;
```

The same wording is on IMapleSkyStrategy.setMetadata.

In the current implementation, protocol admins are the governor and the operational admin (`onlyProtocolAdmins`). The implementation uses `onlyOpsAdminOrStrategyDelegate` instead:

https://github.com/sherlock-audit/2026-09-maple-sept-7th-2026/blob/main/maple-labs__defi-strategies-private/contracts/aaveV3/MapleAaveV3Strategy.sol#L79-L87

https://github.com/sherlock-audit/2026-09-maple-sept-7th-2026/blob/main/maple-labs__defi-strategies-private/contracts/aaveV3/MapleAaveV3Strategy.sol#L248-L256

```
function setMetadata(string calldata metadata_)
    external override whenNotPaused nonReentrant onlyOpsAdminOrStrategyDelegate
    ↪  whenLossNotRecognized
{
    require(keccak256(bytes(metadata_)) != keccak256(bytes(strategyMetadata)),
    ↪  "MAV3S:SM:NO_CHANGE");

    strategyMetadata = metadata_;

    emit StrategyMetadataSet(metadata_);
}
```

Impact

Integrators and operators following the interface will believe only the governor / operational admin can set metadata. The strategy delegate can set it.

Tool Used

Manual Review

Recommendation

Consider changing the specification or aligning with it.

Discussion

0xfarhaan

Fixed in https://github.com/maple-labs/defi-strategies/commit/e889e413d1c96146b371e1474ce18fea8c37e404

0xsimao ** 0xsimao**

Fix in #e889e41 looks good, now the interfaces properly document that `setMetadata()` in the strategies is called by the operational admin or strategy delegate. The allocation strategy is also correct.

Issue L-4: `requestRedeem(0)` spender check is redundant cyclical-WM logic [RESOLVED]

Source: https://github.com/sherlock-audit/2026-09-maple-sept-7th-2026/issues/53

Summary

V500 `MaplePoolManager.requestRedeem` still special-cases `shares_ == 0` so any spender with a nonzero pool allowance can call `requestRedeem(0, owner)`. On a cyclical withdrawal manager, `addShares(0, owner)` postpones that owner's entire `lockedShares` balance. V500 pools use the queue WM, so the branch is dead leftover code.

Vulnerability Detail

The pool forwards the owner's redeem request to the pool manager without reducing allowance when the share amount is zero:

https://github.com/sherlock-audit/2026-09-maple-sept-7th-2026/blob/main/maple-labs__pool-v2-private/contracts/MaplePool.sol#L235-L248

```
if (msg.sender != owner_) {
    _decreaseAllowance(owner_, msg.sender, escrowShares_);
}
// ...
IPoolManagerLike(manager).requestRedeem(escrowShares_, owner_, msg.sender);
```

The pool manager then only requires a nonzero allowance for the zero-share case, and still calls `addShares`:

https://github.com/sherlock-audit/2026-09-maple-sept-7th-2026/blob/main/maple-labs__pool-v2-private/contracts/MaplePoolManager.sol#L336-L348

```
function requestRedeem(uint256 shares_, address owner_, address sender_) external
↪  override whenNotPaused nonReentrant onlyPool {
    address pool_ = pool;

    require(ERC20Helper.approve(pool_, withdrawalManager, shares_),
    ↪  "PM:RR:APPROVE_FAIL");

    if (sender_ != owner_ && shares_ == 0) {
        require(IPoolLike(pool_).allowance(owner_, sender_) > 0,
        ↪  "PM:RR:NO_ALLOWANCE");
    }

    IWithdrawalManagerLike(withdrawalManager).addShares(shares_, owner_);
```

```
    emit RedeemRequested(owner_, shares_);
}
```

On a cyclical withdrawal manager, `addShares(0, owner)` moves that owner's existing locked shares to `currentCycle + 2`. A spender who was approved for any amount could postpone the owner's locked exit without consuming allowance. V500 `setWithdrawalManager` only accepts a queue WM factory. The four upgraded pools (syrup USDC, syrup USDT, syrup USDG, secured lending USDC) all use FIFO queue WMs. Queue `addShares(0)` doesn't postpone locked shares, so the branch has no effect on in-scope upgraded pools.

Impact

Any spender with a leftover nonzero allowance can call `requestRedeem(0, owner)` without consuming that allowance. The Pool Manager still forwards `addShares(0, owner)` to the withdrawal manager. On a cyclical withdrawal manager that call postpones the owner's entire `lockedShares` balance to `currentCycle + 2`. The owner misses their current exit window and waits two extra cycles to withdraw. In this specific case, the spender can't steal shares or assets, but they can delay an LP's redemption indefinitely by repeating the call whenever a new window opens.

Tool Used

Manual Review

Recommendation

Consider removing the leftover branch:

```
if (sender_ != owner_ && shares_ == 0) {
    require(IPoolLike(pool_).allowance(owner_, sender_) > 0, "PM:RR:NO_ALLOWANCE");
}
```

Discussion

0xfarhaan

Fixed in https://github.com/maple-labs/pool-v2/commit/a3f77cc4be6d100d02987b500059432cd6c36e3e

0xsimao

Fix in #a3f77cc looks good, the guard is no longer needed for queue managers, and the to be upgraded pools SyrupUSDC, SyrupUSDT, SyrupUSDG and MISL don't use the cyclical manager.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.